

UNIVERSITÉ CHEIKH AHMADOUL KHADIM

Architecture Détaillée : Explications

Réalisé par l'équipe :

Makane Seck

Mor Faye

Mame Amy Lo

Makhtar Wade

Sommaire

1. Le rôle du Auth Service (Service d'Authentification)
2. Le rôle du Student Service (Service Étudiants).
3. Le rôle du GraphQL Gateway (Passerelle GraphQL)
4. Les avantages de GraphQL
5. Les avantages de l'architecture Microservices

Document Explicatif – Projet GraphQL & Microservices

Ce document présente l'architecture globale de notre solution et détaille les rôles de chaque composant, ainsi que les avantages apportés par les technologies choisies, conformément aux exigences du projet.

1. Le rôle du Auth Service (Service d'Authentification)

Le **Auth Service** est un microservice dédié exclusivement à la sécurité et à l'identité. Il agit comme un poste de contrôle indépendant. Ses responsabilités sont de :

- Vérifier les identifiants (nom d'utilisateur et mot de passe) des administrateurs.
- Générer et signer cryptographiquement un passeport numérique (Token JWT) en cas de connexion réussie.
- Vérifier la validité des tokens présentés par les autres services (la Gateway) pour s'assurer qu'ils n'ont pas expiré et n'ont pas été falsifiés.

2. Le rôle du Student Service (Service Étudiants)

Le **Student Service** est le microservice métier. Il est spécialisé dans une seule tâche : la gestion de la ressource "Étudiant". Il ignore tout de l'authentification (ce n'est pas son travail). Ses responsabilités sont de :

- Gérer les opérations CRUD (Créer, Lire, Mettre à jour, Supprimer) sur les étudiants.
- Supprimer des étudiants en lot (Batch Delete).

- Fournir un moteur de recherche interne pour filtrer les étudiants selon des critères précis (nom, matricule, email, etc.).
- Conserver l'intégrité de la base de données des étudiants.

3. Le rôle du GraphQL Gateway (Passerelle GraphQL)

La **Gateway GraphQL** est le chef d'orchestre de l'architecture. C'est le composant le plus important côté client, car c'est le seul avec lequel ce dernier communique. Ses responsabilités sont de :

- **Fournir un point d'entrée unique** : Au lieu d'appeler de multiples adresses REST, le client n'envoie ses requêtes qu'à une seule adresse (`/graphql`).
- **Orchestrer et relayer (Proxy)** : Elle lit les requêtes GraphQL et va chercher les données nécessaires en interrogeant (via requêtes HTTP internes) le Student Service.
- **Sécuriser les accès** : Avant d'exécuter une action sensible, la Gateway extrait le Token JWT de la requête du client, l'envoie au Auth Service pour validation, et bloque la requête si le token est invalide.

4. Les avantages de GraphQL

L'introduction de GraphQL résout les limites inhérentes à notre précédente architecture REST :

- **Fini l'Over-fetching (Récupération excessive)** : Le client demande uniquement les champs dont il a besoin. Si une application mobile n'a besoin que des ``noms`` et ``prenoms``, le réseau n'est pas encombré par les ``emails`` ou ``matricules``.

- **Fini l'Under-fetching (Requêtes multiples)** : GraphQL permet d'agréger plusieurs sources de données en une seule requête, évitant d'enchaîner de multiples requêtes HTTP pour afficher une seule page.
- **Typage fort et auto-documentation** : Le schéma GraphQL agit comme un contrat strict. Les développeurs front-end savent exactement quelles données sont disponibles sans avoir à lire une longue documentation externe.

5. Les avantages de l'architecture Microservices

La transition d'une application monolithique vers une architecture en microservices offre des avantages majeurs :

- **Tolérance aux pannes (Résilience)** : Si le Student Service subit une erreur ou tombe en panne, le Auth Service continue de fonctionner. Le système entier ne s'effondre pas.
- **Évolutivité (Scalabilité ciblée)** : Si le composant de gestion des étudiants reçoit énormément de trafic lors des inscriptions, nous pouvons allouer plus de mémoire *uniquement* au Student Service, sans toucher au Auth Service.
- **Indépendance des technologies** : Bien que tout soit en Node.js ici, chaque microservice pourrait à l'avenir être réécrit dans le langage le plus adapté (ex: Python pour l'IA, Go pour la performance) sans affecter le reste du système.
- **Déploiement facilité** : Une équipe peut mettre à jour le système d'authentification sans avoir à redéployer ou risquer de casser la gestion des étudiants.